

Final Project Report: Random Dungeons

Muhammad Ansar Ali Adnan

May 11, 2026

Contents

1	Introduction	3
2	System Architecture & Core OOP	3
3	Data Structures & Algorithms	5
4	Advanced C++ Features	5
5	Challenges & Solutions	7
6	Test Cases	7
7	Edge Cases	8
8	Future Enhancements	8
9	Conclusion	8

1 Introduction

We made a procedural dungeon generator game where the main goal was to generate a top-down 2D dungeon with an entrance and a staircase leading up, kinda like the Persona 3 dungeon system.

The player starts at the entrance, and their goal is to survive each floor, reach the 5th floor, and conquer the dungeon.

When you enter a new floor, there's fog everywhere hiding the map layout. As you explore, the tiles right next to you (up, down, left, right, diagonally) get unfogged (the internal state flips from 1 for fogged to 0 for unfogged).

There are monsters with AI to chase you when you get close, plus chests and items like Reveal Potions to help clear the map. The layout is totally random, so it can generate narrow branching paths or wide-open arenas on the fly.

Combat is live—no turn-based stuff. You don't even have to fight the enemies; you can just run for the stairs if you want. Reaching the stage 5 stairs gets you the win. We also implemented Normal and Hard modes, which dictate how the fog behaves when you move away from tiles you've already revealed.

2 System Architecture & Core OOP

Our game is written in C++17 and SFML manages the front-end. We leaned heavily into OOP to keep the game modular and memory-safe.

- **Classes & Objects:** We split the system into logical parts. `Game` handles the main engine loop and rendering, while `DungeonFloor` handles the stage generation and fog.
- **Encapsulation:** We kept variables protected to prevent weird bugs. For example, `healthPoints` in the `Character` class is `protected`, so you have to use the `takeDamage()` function instead of messing with the health pool directly.
- **Inheritance & Polymorphism:** `Entity` is our main parent class. `Item` and `Character` inherit from it, and `Player` and `Enemy` inherit from `Character`. This let us use polymorphism to easily loop through all entities and update them at once.
- **Abstract Classes:** `Entity` acts as a strict blueprint. We made it an abstract class with a pure virtual `draw()` function, forcing all child classes to handle their own rendering.

```
1 class Entity {
2     protected:
3         Point position;
4         sf::Sprite sprite;
5     public:
6         // Pure virtual function enforcing rendering
7         virtual void draw(sf::RenderWindow& window) = 0;
8 };
```

Listing 1: Making Entity an Abstract Class

- **Operator Overloading:** We overloaded the `==` operator in our `Point` struct so we could just compare coordinates directly instead of checking `x` and `y` values manually everywhere.

3 Data Structures & Algorithms

- **Algorithm 1: Dijkstra's Pathfinding:** We used Dijkstra's algorithm to handle the enemy AI. Applying what we learned in discrete math to trace the shortest unblocked path to the player was honestly pretty fun and made the enemies much smarter.
- **Data Structure 1: Priority Queue:** To make Dijkstra fast, we used a `std::priority_queue`. It automatically sorts the map nodes by distance, so the AI always checks the closest tiles first.

```
1 // Distance tracking and Priority Queue setup
2 distances[position.y][position.x] = 0;
3 std::priority_queue<Node, std::vector<Node>, std::greater<Node>> pq;
4
5 bool canSeePlayer = inVisionRange(position, playerPos, 6);
6 bool pathFound = false;
7
8 if (canSeePlayer) {
9     pq.push({position, 0});
10    // Pathfinding evaluation loop continues...
```

Listing 2: Priority Queue setup for Enemy AI

- **Data Structure 2: 2D Arrays:** The dungeon grid and fog states are handled using massive static 2D arrays (`grid[maxHeight][maxWidth]`). This gives us quick $O(1)$ constant time lookups for movement.
- **Data Structure 3: Dynamic Arrays (`std::vector`):** To manage collections of entities that change size while the game is running, we heavily utilized vectors. For example, `std::vector<Point> pathHistory` tracks the random walker's exact steps so we can guarantee the stairs spawn at the very end of a valid path. Vectors also manage our `activeEnemies` and `floorItems`, letting us seamlessly `.erase()` a potion when the player uses it.

4 Advanced C++ Features

- **Templates:** We built a template function called `isTileOccupied`. Earlier we had written separate functions for various classes, but templates saved us from writing a bunch of duplicate code. It lets us pass in vectors of any object type (like `Items` or `Enemies`) to check for collisions.

```
1 template <typename T>
2 bool isTileOccupied(const std::vector<std::unique_ptr<T>>& entityList,
3 Point target, const Entity* ignoreEntity = nullptr) {
4     for (const auto& entity : entityList) {
5         if (entity.get() != ignoreEntity && entity->getPosition() ==
6 target) {
7             return true;
8         }
9     }
10    return false;
```

Listing 3: Template Function for Collision Checks

- **STL Containers & Smart Pointers:** To stop memory leaks when floors reset, we exclusively used `std::unique_ptr` inside our vectors. The memory just cleans itself up automatically.

5 Challenges & Solutions

- **Challenge 1:** Cross-platform struggles.
- **Solution 1:** Ali and I ran into the classic "it works on my machine" problem that most devs run into. As we work on Linux, our compiled file was running on our machines perfectly, but when we tried it on Windows machines, there were DLL errors. We bypassed this by compiling the code natively on the Windows machines using SFML and our standard Makefile.
- **Challenge 2:** Enemy stacking and dumb AI.
- **Solution 2:** We originally had a dummy AI where enemies moved randomly and kept stacking on top of each other. On Abdullah's recommendation, we fixed this by upgrading to Dijkstra's algorithm and using our template collision function to force enemies to treat each other as solid walls.
- **Challenge 3:** The Frame-Rate Counter Overflow Bug.
- **Solution 3:** We hit a crazy bug where stepping on the exit stairs would cause the `floorLevel` counter to skyrocket into the millions. We realized this was a game loop issue: because the loop runs dozens of times a second, the `floorLevel++` instruction was firing repeatedly before the new floor could render. We fixed it by strictly managing the game state, making sure `resetDungeon()` immediately overrides the player's position so the collision condition breaks before the next frame.

6 Test Cases

- **Feature:** Level Generation. **Test Description:** Start the game and observe the initial floor layout. **Expected Result:** The player should spawn at the ENTRANCE, and a path must exist to the STAIRS. All other tiles start hidden.
- **Feature:** Fog of War. **Test Description:** Move the player in a cardinal direction. **Expected Result:** As the player moves, a grid of tiles around the player's new position must become visible.
- **Feature:** Wall Collision. **Test Description:** Attempt to move the player into a WALL tile. **Expected Result:** The player's position should not change; the movement command should be ignored.
- **Feature:** Monster Collision. **Test Description:** Walk directly into a monster. **Expected Result:** The interface must display damage, the screen flashes red, and health drops. If dead, "GAME OVER" state triggers.
- **Feature:** Reveal Potion. **Test Description:** Pick up a "Reveal Potion". **Expected Result:** The `activeMessage` changes to "Map Revealed!" and every single tile on the current floor becomes visible immediately.
- **Feature:** Floor Progression. **Test Description:** Walk onto the STAIRS. **Expected Result:** The `floorLevel` counter increments, and a completely new dungeon layout generates.

- **Feature:** Win Condition (Cap). **Test Description:** Reach and enter the STAIRS on Floor 5. **Expected Result:** The game transitions to the WIN state. The success message appears, and movement is disabled.

7 Edge Cases

1. **The "Trap" Edge Case:** Testing if a monster can spawn on top of the player. Our code prevents this by checking the spawn distance is > 4 tiles away.
2. **The "Stuck Walker" Edge Case:** Testing if the procedural generation breaks array bounds. The walker uses bound checks (> 1 and $< MAX - 2$) to ensure it never digs outside the grid, preventing crashes.
3. **The "R" Key Reset:** Testing if pressing 'R' works correctly. The code isolates state checks (isGameComplete, playerDead) before allowing the 'R' key to trigger a reset so you don't accidentally restart mid-game.
4. **Item Persistence:** Testing if potions disappear after use. We used the `erase()` function on the `floorItems` vector to ensure a potion can only be consumed once.

8 Future Enhancements

While the core engine is solid, future versions of this project could include:

- **Complex Weapon Systems:** Implementing an inventory queue and specialized weapons.
- **Environmental Hazards:** Adding spike traps or arrow dispensers that use the same collision and damage systems currently affecting the player.
- **Sprite Animation:** Upgrading from static rendering to animated sprite sheets using SFML's texture rect functions to show movement and attacks.

9 Conclusion

We successfully pushed the game way past the rudimentary version from the last milestone. We hit all the core OOP requirements, got real graph-based enemy AI working, and ended up with a fully playable procedural dungeon crawler that reflects exactly what we originally envisioned.